

GitHub: the terms you actually need

A survival glossary for the Ocean Hackathon. You already know version control and CI as concepts; this is the vocabulary, the exact commands, and the errors that will bite you.

The mental model, in four sentences

A **repository** is a project folder whose entire history is recorded. A **commit** is one saved snapshot of that folder with a message explaining what changed. Your computer has a full copy of the history and so does GitHub's server; **push** and **pull** move commits between them. Everything else in this document is detail hanging off those three ideas.

The thing that trips people up: git and GitHub are different. **Git** is the version-control program running on your laptop, invented before GitHub existed and usable entirely offline. **GitHub** is a website that hosts git repositories and adds issues, reviews and CI on top. You can use git without GitHub. You cannot really use GitHub without git.

Core vocabulary

repository (repo)	The project and its full history. Yours will be public so judges can see it.
commit	One snapshot plus a message. Think of it as a save point you can return to and explain. Commit messages are written for your future self at 2am.
branch	A parallel line of commits. <code>main</code> is the default. Branches let two people work without overwriting each other.
main	The primary branch. Older repos and tutorials call it <code>master</code> ; same thing, renamed in 2020.
clone	Download a full copy of a repo, history included. Done once per machine.
remote	A copy of the repo living somewhere else, normally on GitHub.
origin	The conventional nickname for your main remote. Not a keyword, just a habit.
push	Send your local commits up to the remote. Nothing on GitHub changes until you push.
pull	Fetch commits from the remote and merge them into your local copy. Do this before you start work if a teammate has been pushing.
fetch	Download remote commits without merging them yet. The cautious half of a pull.
staging area (index)	The waiting room. <code>git add</code> puts changes in it; <code>git commit</code> turns whatever is in it into a snapshot. This two-step surprises everyone at first.
merge	Combine two branches. Usually automatic.
merge conflict	Both branches changed the same lines and git will not guess. It marks the file with <code><<<<<<</code> and <code>>>>>>></code> ; you edit it by hand, delete the markers, then add and commit.
pull request (PR)	A proposal to merge one branch into another, with room for discussion and review. GitHub's invention, not git's. The review gate in your process.
.gitignore	A list of files git should not track. Yours excludes data files and Python caches, deliberately keeping the figures so the README renders.
README.md	Rendered automatically on the repo's front page. Written in Markdown. This is your shop window.
fork	Your own server-side copy of somebody else's repo. Use it to contribute to projects you cannot push to.

The commands you will actually type

First time only, on a new machine:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Publishing this project for the first time:

```
cd mhw-phyto
git init -b main          # start tracking this folder
git add .                 # stage everything
git commit -m "initial pipeline"
git remote add origin https://github.com/USER/REPO.git
git push -u origin main   # -u remembers the target for next time
```

The everyday loop, which is about 90 percent of all git use:

```
git status                # what have I changed? use this constantly
git pull                  # get teammates' work first
git add -A                # stage all changes
git commit -m "add NPZD counterfactual"
git push                  # -u was set, so no arguments needed
```

Working on a feature without disturbing main:

```
git switch -c track-c-npzd # create branch and move onto it
...edit, add, commit...
git push -u origin track-c-npzd
# then open a pull request on the GitHub website
```

Undoing things, in ascending order of violence:

```
git restore FILE          # discard uncommitted edits to one file
git restore --staged FILE # unstage but keep the edits
git commit --amend         # fix the last commit message
git revert COMMIT          # new commit undoing an old one (safe, shared)
git reset --hard COMMIT   # DESTROYS later work. Last resort.
```

Rule of thumb: **revert** is safe on anything you have pushed, because it adds history. **reset --hard** deletes history and will confuse everyone who already pulled it.

Touring the GitHub website

Code tab	The file browser, with the rendered README underneath. Where visitors land.
Commits	The history. Click any commit to see exactly which lines changed.
Issues	Task tracker. One issue per workstream track is a cheap, visible backlog.
Pull requests	Open proposals and their reviews.
Actions	Your CI. Every push triggers the workflow; each run gets a page with logs.
Settings	Where secrets, collaborators and visibility live. Only owners see it.
green tick / red X	Beside a commit: whether CI passed. Judges notice this.

Terms specific to what we built

GitHub Actions	GitHub's built-in CI. Free for public repos, within monthly minutes.
workflow	A YAML file in <code>.github/workflows/</code> describing an automated job. You have two: <code>pipeline.yml</code> and <code>fetch-cmems-data.yml</code> .
runner	The throwaway virtual machine that executes a workflow. Freshly built each run, destroyed afterwards, and crucially it has unrestricted internet.
job / step	A workflow has jobs; a job has ordered steps. One failing step fails the job.
trigger (on :)	What starts a workflow. Yours run on push, on pull request, or manually.
workflow_dispatch	The trigger that adds a Run workflow button with input fields. This is how you launch the Copernicus download and pass it a region and dates.
secret	An encrypted value stored in repo Settings, exposed to workflows as an environment variable and masked in logs. Where your Copernicus username and password belong. Never in a commit, never in a chat window.
artifact	A file a workflow produces and offers for download from the run page. Your figures come out this way, and so will the downloaded ocean data.
badge	The small status image in the README, served live by GitHub. Shows whether the latest run passed.
Colab badge	Opens the notebook in Google Colab, which clones the repo and runs it in the browser. Your zero-install demo.

How this maps to process vocabulary you already have

branch + PR	The review gate. A PR is a formal request for peer review with the diff attached, and can be made a required step before merge.
CI on every push	Continuous integration in the literal sense: the build and test suite runs against the integrated trunk automatically, so breakage surfaces in minutes.
Issues + labels	A lightweight backlog. Six issues, one per track, gives visible WIP.
protected main	Definition of done, enforced mechanically: Settings, Branches, require the status check to pass before merge. Nobody can push a red build to trunk.
commit granularity	One logical change per commit. The history becomes the audit trail, which is the whole point when a judge asks who validated what.

Errors you will hit, and what they actually mean

Support for password authentication was removed	GitHub wants a personal access token instead of your account password. Settings, Developer settings, Personal access tokens, Fine-grained, repo scope. Or install the gh CLI and run <code>gh auth login</code> , which handles it for you.
Updates were rejected... fetch first	The remote has commits you do not. Run <code>git pull</code> , resolve anything conflicting, push again. Never reach for <code>--force</code> on a shared branch.
refusing to merge unrelated histories	You ticked 'add a README' when creating the repo, so both sides have independent roots. Easiest fix: delete the GitHub repo, recreate it empty, push again.
fatal: not a git repository	You are in the wrong directory, or never ran <code>git init</code> .

Workflow file missing after web upload	Some browsers silently skip dotfiles when you drag a folder in. Recreate it with Add file, Create new file, and type the full path <code>.github/workflows/pipeline.yml</code> as the filename.
Committed a secret by mistake	Assume it is compromised the moment it is pushed; deleting the commit is not enough. Rotate the credential immediately, then clean the history.

Safe to ignore for now

rebase, cherry-pick, stash, submodule, tag, squash, reflog, bisect, sparse checkout. All useful eventually, none needed to ship this weekend. If someone at the venue tells you that you must rebase, you may smile and keep merging.

One honest caveat: GitHub moves its menus around. Exact paths in Settings may differ slightly from the descriptions above.